

BOX 2026 - Homework 3

Joachim Le Fournis

2.

Let's order our set of strings $\mathcal{S}$ as $\{s_1, s_2, \dots, s_n\}$, and have a few helper function from the previous question :

$\text{ov}(s_i, s_j)$ = length of the longest suffix of s_i which is a prefix of s_j

$\text{merge}(s_i, s_j) = s_i \cdot s_j[\text{ov}(s_i, s_j) :]$ a function that merge two strings like previously

Our weighted directed graph $G = (V, E, w)$ will be :

- $V = \{0, 1, \dots, n\}$ where each node $i \geq 1$ represents s_i and 0 is the virtual source used for the reduction
- $E = \{(0, i) \mid \forall i \in [1; n]\} \sqcup \{(i, j) \mid \forall i, j \in [1; n], i \neq j\}$
- $\forall i \in [1; n], w(0, i) = |s_i|$ and $\forall i, j \in [1; n]^2$ where $i \neq j, w(i, j) = |s_j| - \text{ov}(s_i, s_j)$

When we have $(0, v_1, \dots, v_n)$ a solution of open-TSP in this graph G , we can reconstruct the superstring as such : $s = \text{merge}(\dots \text{merge}(\text{merge}(s_{v_1}, s_{v_2}), s_{v_3}), \dots, s_{v_n})$

3.

One first remark we can make is that $|s| = |s_{v_1}| + \sum_{k=1}^{n-1} (|s_{v_{k+1}}| - \text{ov}(s_{v_k}, s_{v_{k+1}}))$ by definition of the merge function.

However, by definition of solution to open-TSP, $|s| = w(0, v_1) + \sum_{k=1}^{n-1} w(v_k, v_{k+1})$.

Firstly, we know that the superstring s is built using only merge on its substring, thus it is obviously a valid superstring/solution to the SCS problem.

Thus, we only need to prove that

$$\text{opt}_{\text{SCS}}(S) = \min_{\text{Hamiltonian path}} \left(w(0, v_1) + \sum_k w(v_k, v_{k+1}) \right)$$

- $\leq$: let's suppose that this isn't the case, and we have an hamiltonian path whose right hand side of the previous equality is smaller than $\text{opt}_{\text{SCS}}(S)$. Then, we can construct as explained in the previous question a new superstring that is shorter than $\text{opt}_{\text{SCS}}(S)$, which means we didn't have the optimal solution : absurd.
- $\geq$: Let t be an optimal solution to the SCS problem, i.e. $|t| = \text{opt}_{\text{SCS}}(S)$. For each $s_i \in S$, let p_i denote the starting position of its leftmost occurrence in t . Let $v_1, \dots, v_n$ be the strings of S sorted by increasing p_{v_i} (ties broken arbitrarily).

Let's prove by absurd that this order also respects the ending positions, i.e.

$$p_{v_k} + |s_{v_k}| \leq p_{v_{k+1}} + |s_{v_{k+1}}| \quad \text{for all } k.$$

The occurrence of $s_{v_{k+1}}$ starts after (or at) the start of s_{v_k} 's occurrence but ends before it, i.e. it lies entirely inside s_{v_k} 's occurrence in t . This means $s_{v_{k+1}}$ occurs as a substring of s_{v_k} , i.e. $s_{v_{k+1}} \subset s_{v_k}$, contradicting the hypothesis $\forall (u, v) \in S, u \not\subset v$.

So the occurrences of $s_{v_1}, \dots, s_{v_n}$ form a genuine left-to-right chain inside t . Let $\text{step}_k = p_{v_{k+1}} - p_{v_k} \geq 0$. If $\text{step}_k \leq |s_{v_k}|$, the two occurrences overlap inside t over a region of length $|s_{v_k}| - \text{step}_k$,

which is by construction a common suffix of s_{v_k} and prefix of $s_{v_{k+1}}$ – hence, since ov is by definition the **maximal** such length,

$$ov(s_{v_k}, s_{v_{k+1}}) \geq |s_{v_k}| - \text{step}_k, \quad \text{i.e.} \quad \text{step}_k \geq |s_{v_k}| - ov(s_{v_k}, s_{v_{k+1}}).$$

If instead $\text{step}_k > |s_{v_k}|$ (no overlap), the same inequality holds trivially since $ov(s_{v_k}, s_{v_{k+1}}) \leq |s_{v_k}|$ always. So in either case

$$\text{step}_k \geq |s_{v_k}| - ov(s_{v_k}, s_{v_{k+1}}) \quad \text{for all } k.$$

Summing over $k = 1, \dots, n-1$, we get by telescope sum

$$p_{v_n} - p_{v_1} = \sum_{k=1}^{n-1} \text{step}_k \geq \sum_{k=1}^{n-1} |s_{v_k}| - \sum_{k=1}^{n-1} ov(s_{v_k}, s_{v_{k+1}}).$$

Adding $|s_{v_n}|$ to both sides and re-indexing the length sum on the right ($\sum_{k=1}^{n-1} |s_{v_k}| + |s_{v_n}| = |s_{v_1}| + \sum_{k=1}^{n-1} |s_{v_{k+1}}|$) allows us to get to

$$p_{v_n} + |s_{v_n}| - p_{v_1} \geq |s_{v_1}| + \sum_{k=1}^{n-1} (|s_{v_{k+1}}| - ov(s_{v_k}, s_{v_{k+1}})) = w(0, v_1) + \sum_{k=1}^{n-1} w(v_k, v_{k+1}).$$

Since $p_{v_1} \geq 0$ and the occurrence of s_{v_n} lies inside t (so $p_{v_n} + |s_{v_n}| \leq |t|$), the left-hand side is at most $|t|$, thus

$$|t| \geq w(0, v_1) + \sum_{k=1}^{n-1} w(v_k, v_{k+1}) \geq \min_{\text{Hamiltonian path}} \left(w(0, v_1) + \sum_k w(v_k, v_{k+1}) \right).$$

As this holds for t optimal, i.e. $|t| = \text{opt}_{\text{SCS}(S)}$, we get

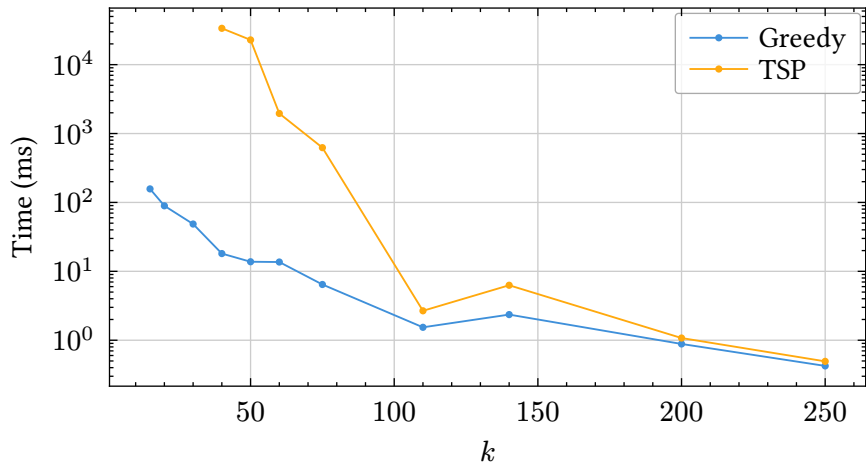
$$\text{opt}_{\text{SCS}(S)} \geq \min_{\text{Hamiltonian path}} \left(w(0, v_1) + \sum_k w(v_k, v_{k+1}) \right).$$

5.

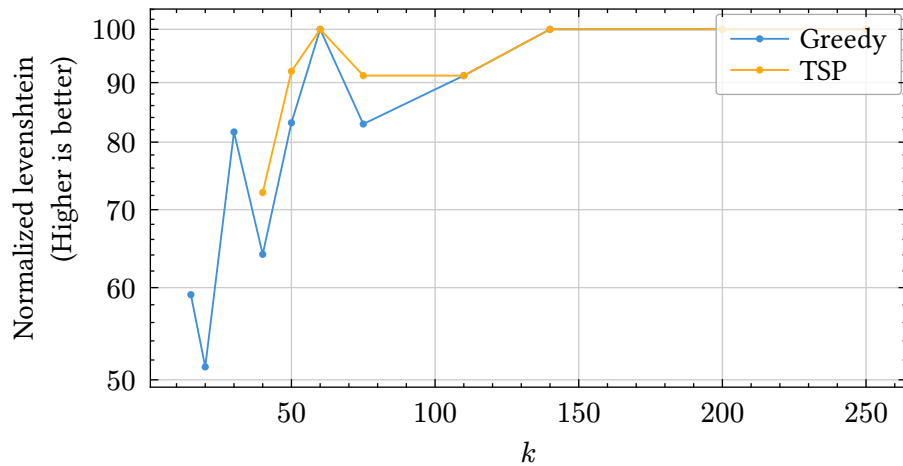
First thing we can notice is that the impact of k is only not directly correlated to the results : for instance, the complex part isn't generating the distance matrix (which depends of k), but rather the number of reads n made. We can compute that however, k and n are linked in probability : $\mathbb{E}[n] = \frac{2L}{k+1}$. Thus, to get more precise results, we tried to repeat the same experiment with different seed and then take the mean of time and quality of result.

Speaking of that, to measure the quality of the result, we did chose to use the levenshtein distance. It is probably not the perfect solution, but at least it gives idea of how far away the result is away from the base genome. To be able to compare it at different k value, we did normalize it like so $\text{normalized}(a, b) = 1 - \frac{\text{levenshtein}(a, b)}{\max(|a|, |b|)}$, to be able to get a result between 0% and 100%, and higher is better.

Also, we couldn't run the algorithm on the whole genome, which was about a few megabytes, because it generated way to many reads, which caused way too long overlap check. We chose to only use about 400bp for this exercise. Moreover, our TSP solver is using dynamic programming, which caused it to often crash due to Out-of-Memory. We limited in our experiment to only run it when we have less than 19 reads to avoid crashing our computer.



As expected, when k grows, n get smaller and thus the algorithm get faster and faster. Also expected, whenever k is big or small, the TSP solution is very slow compared to the greedy algorithm.



We can also notice that when k is getting higher, our result is getting better and better. Starting at $k = 140$, we're even getting the exact same genome as the one we started with (which can be expected because of the small part of the genome we worked on). Also, the TSP solution is never worse on these exemples than the greedy one, which was expected too.

On both of these figures, we can also notice that all data points are not perfectly following what we expected: the reads are random, we will get for some high values of k more reads than for some lower ones, which directly impacts our algorithms.